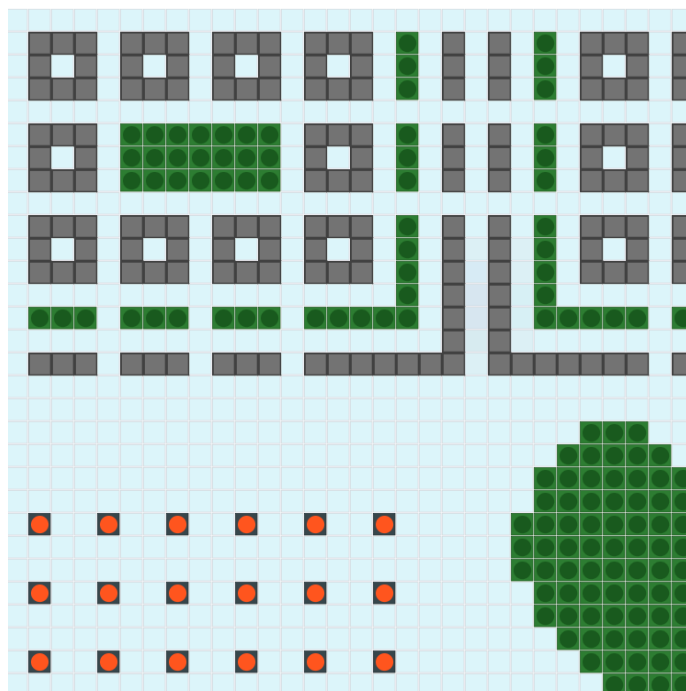


Rapport Projet Génie Logiciel CyBreathe



Sommaire

I. Introduction et Contexte.....	3
1. Présentation générale du projet.....	3
2. Rôles et apports.....	3
3. Planification.....	3
II. Problèmes & Solutions.....	4
1. Problèmes rencontrés.....	4
2. Solutions apportées.....	4
3. Limites.....	4
III. Architecture technique du projet.....	5
1. Choix de l'architecture MVC.....	5
2. Test unitaire.....	5
3. Système de Sauvegarde.....	6
4. Environnement Technique.....	6
IV. Fonctionnement du Moteur de Simulation.....	7
1. Règle de l'automate cellulaire.....	7
A. Voisinage et gestion des limites.....	7
B. Modélisation de l'influence du vent.....	7
C. Comportement et lois de transition spécifiques des cellules.....	8
2. Mécanisme de mise à jour.....	9
3. Ajout simplifié de type de cellule.....	9
V. Conclusion.....	11
VI. Annexes.....	12
1. Diagramme de cas d'utilisation.....	12
2. Diagramme de classe.....	13

I. Introduction et Contexte

1. Présentation générale du projet

Le thème choisi pour notre projet est la simulation de cellule dans un plan 2D. Dans l'étude de ce projet, nous nous sommes intéressés à la propagation de la pollution atmosphérique. Notre application est constituée de différents types de cellule, certaines produisent de la pollution tandis que d'autres absorbent cette pollution.

L'application permet de visualiser la pollution atmosphérique des dioxyde de soufre et de savoir si une usine ou tout autre source de pollution peut être nocive à la population ou comment nous pouvons réduire effectivement la concentration en dioxyde de soufre dans l'air.

Nous pouvons donc augmenter certains paramètres comme la pollution ou l'absorption de la pollution pour modéliser des usines plus ou moins polluantes et des types de végétations plus ou moins efficaces pour absorber certains types de pollutions.

2. Rôles et apports

La répartition des rôles au sein du projet CyBreathe s'organise selon l'architecture PAC afin de segmenter strictement les développements.

Antonin, développeur principal backend, a conçu le moteur algorithmique d'automate cellulaire synchrone (Grid, Simulation), codé les lois physiques de diffusion-advection (AirCell, FactoryCell, etc.), et implémenté le mécanisme de double tampon (AbstractCell) ainsi que la sérialisation binaire .cyp. Mathieu, architecte logiciel, a réalisé l'intégralité de la modélisation fonctionnelle (diagrammes de cas d'utilisation et de classes), défini la structure MVC et développé le lanceur racine CyBreathe avec son mécanisme de repli vers la console.

Louis s'est chargé de l'interface graphique (GUI) sous JavaFX, concevant l'ossature visuelle (SimulationView), les curseurs de contrôle, la boussole du vent et les graphiques statistiques dynamiques. Rayan a géré le développement événementiel et les outils de dessin, programmant le rendu des cellules (CellView), la capture des interactions souris (GridView, GridController), les modes pinceau/zone et l'overlay de débogage. Enfin, Mathéo a développé l'interface en ligne de commande (CommandLineInterface, CLIController) avec rendu ASCII gradué, tout en assurant la validation via les tests unitaires (CyBreatheTest) et l'intégration continue GitHub Actions.

3. Planification

Afin de répondre aux exigences du projet, il est nécessaire de planifier nos tâches dans l'objectif de la réussite de ce projet. Pour cela, nous avons utilisé divers moyens de communication comme teams, discord et github pour suivre l'avancée du code.

En ce qui concerne la planification, nous nous sommes concertés afin de réfléchir au thème de notre projet en implémentant la simulation de cellule dans un plan 2D avant même la publication du cahier des charges techniques. Lorsque le cahier a été publié, nous avons enfin eu la possibilité de commencer le code. Chaque semaine à compter de la semaine du 18 mai, nous nous réunissons pour discuter des avancements du projet puis nous les présentons à notre tuteur la même semaine. Cela nous a donc permis de progresser de manière efficace et rigoureuse dans notre projet.

II. Problèmes & Solutions

1. Problèmes rencontrés

Il est très compliqué de simuler les comportements et les mouvements de l'air et de la pollution dans un plan 2d de manière précise, nous avons donc opté pour une vision plus simpliste, ici chaque cellule vient regarder les cases adjacentes de la grille constituant un voisinage de Moore.

2. Solutions apportées

La propagation de la pollution a été restreinte au voisinage immédiat de Moore (8 cellules adjacentes). Ce choix méthodologique permet de réduire la complexité algorithmique pour l'ensemble de la grille. Bien que simplifiée, elle ne correspond pas à la réalité.

3. Limites

Le réalisme de la simulation est contraint par des modélisations géométriques absolues. Les obstacles urbains bloquent intégralement les flux de pollution (100 %), omettant les phénomènes de contournement vertical en trois dimensions et les turbulences aérodynamiques. De même, les cellules de végétation possèdent une capacité d'absorption infinie, sans intégration de seuil de saturation biologique face à de fortes concentrations.

Le modèle souffre d'une absence de correspondance physique explicite. La surface réelle représentée par une cellule (arbre isolé, alignement urbain ou forêt dense) n'est pas définie. Ce manque d'échelle géométrique empêche tout calibrage rigoureux ou comparaison quantitative avec des données réelles de stations de surveillance.

Le vecteur vent est appliqué de manière uniforme et unidirectionnelle sur toute la matrice, ce qui est cohérent à l'échelle d'un micro-quartier mais néglige les déviations topographiques locales sur de grands périmètres. Enfin, les dimensions restreintes de la grille (30x30) induisent des effets de bord significatifs aux frontières du domaine simulé.

III. Architecture technique du projet

1. Choix de l'architecture MVC

Pour structurer l'application, nous avons opté pour une architecture Modèle-Vue-Contrôleur (MVC). Ce choix se justifie par la nécessité de découpler entièrement l'état logique de la simulation de sa représentation visuelle.

L'application est ainsi divisée en 3 packages :

Le Modèle (models) : Il encapsule les données et la logique métier de la simulation (la grille, les paramètres physiques, le vent, l'état et le comportement des différentes cellules).

- Le Modèle (models) : Il encapsule les données et la logique métier de la simulation (la grille, les paramètres physiques, le vent, l'état et le comportement des différentes cellules).
- La Vue (view) : Elle est responsable de l'affichage et de l'interaction avec l'utilisateur. Le projet intègre une double implémentation : une interface graphique moderne (JavaFX) et une interface textuelle en ligne de commande (CLI).
- Le Contrôleur (controller) : Il agit de manière à faire communiquer les "view" et les "models" sans que les "view" ne puissent directement discuter avec les "models" . Il reçoit les actions de l'utilisateur via les vues, applique les changements correspondants sur le modèle, et demande aux vues de se rafraîchir lorsque le modèle évolue.

Avec cette architecture toute la logique métier est complètement indépendante des technologies d'affichage. Les classes coeur de notre automate cellulaire ne dépendent pas des interfaces JavaFX/CLI. Cette séparation nous offre deux avantages fondamentaux pour notre projet :

- Portabilité : La logique de simulation du comportement de la pollution est universelle. On peut basculer d'une interface graphique à une console (ou même imaginer une future version web) sans modifier une seule ligne du modèle métier.
- Haute Testabilité (Tests Unitaires) : N'ayant aucune attache avec l'interface graphique, toute la logique algorithmique (diffusion de la pollution, calcul du vent, comportement des usines) peut être validée à l'aide de tests automatisés.

2. Test unitaire

Grâce au découplage imposé par le MVC, le projet intègre une suite complète de tests unitaires via junit & gradle. Ces tests s'exécutent lors de la compilation de notre exécutable afin de vérifier que les règles définies pour notre automate logique et ces cellules.

La validation se concentre sur les briques critiques du modèle :

- Les mathématiques de la simulation : Vérification des algorithmes de diffusion de la pollution entre les cellules et de l'influence vectorielle du vent (advection).
- Le comportement des structures : Validation du polymorphisme des cellules (l'absorption par la végétation, l'étanchéité des bâtiments, les cycles d'émission des usines).

- La cohérence de la grille : Algorithme de calcul du voisinage de Moore (gestion des cas spécifiques des bords et des coins de la grille)

3. Système de Sauvegarde

Pour répondre au besoin de sauvegarder l'état d'une simulation et de la recharger ultérieurement, nous avons choisi d'utiliser la sérialisation binaire native de Java.

Cela se traduit par le fait que toutes nos classes métier (dans le package `models`) utilisent l'interface "Serializable" de Java et possèdent un identifiant de version (`serialVersionUID`). Ce qui sécurise la compatibilité des fichiers de sauvegarde lors des futures évolutions du code.

Fonctionnement de l'Export / Import :

- En écriture, l'application capture l'état complet du modèle et l'enregistre dans un fichier binaire ".cyp" via la classe Java `ObjectOutputStream`.
- En lecture, le fichier est désérialisé pour reconstruire l'objet `Simulation` à chaud en mémoire, qui est ensuite fourni aux contrôleurs pour mettre à jour l'affichage de l'utilisateur.

4. Environnement Technique

- Java 21 (JDK 21)
- Gradle : Outils pour centraliser la gestion des dépendances

Procédures pour exécuter le projet, pour faciliter le travail de développement et la livraison du projet, trois commandes principales ont été configurées via le wrapper Gradle (`gradlew` ou `gradlew.bat` (pour Windows)) :

- Mode Développement : `./gradlew run`
- Compilation et Packaging : `./gradlew build`
- Exécution Autonome : `java -jar build/libs/CyBreathe-1.0-SNAPSHOT.jar`

IV. Fonctionnement du Moteur de Simulation

Le cœur algorithmique de CyBreathe repose sur les principes des automates cellulaires dans une grille 2D

1. Règle de l'automate cellulaire

A. Voisinage et gestion des limites

Pour évaluer les interactions environnementales et la propagation des polluants, le moteur s'appuie sur un voisinage de Moore d'ordre 1 à 8 directions. Ainsi, pour une cellule donnée aux coordonnées (x, y), l'algorithme analyse l'état des huit cellules adjacentes (Nord, Sud, Est, Ouest et les quatre diagonales).

Les cellules situées en dehors des limites physiques de la grille (au niveau des bords et des coins de la matrice) sont systématiquement ignorées. Seuls les voisins valides inclus dans l'intervalle $[0, \text{largeur}-1]$ $[0, \text{hauteur}-1]$ participent aux calculs.

B. Modélisation de l'influence du vent

Le vent introduit une force physique vectorielle directionnelle qui modifie la diffusion naturelle de la pollution. Il est défini globalement par un vecteur de direction $W = (W_x, W_y)$ et une intensité s in $[0.0, 1.0]$.

Le vent détermine un coefficient de pondération spécifique (un poids) pour chaque cellule voisine :

1. En l'absence de vent (force nulle ou direction non configurée), le poids par défaut de chaque voisin est égal à 1.0.
2. En présence de vent, l'algorithme calcule le vecteur de déplacement allant du voisin vers la cellule courante :

$$v = (x_{\text{courant}} - x_{\text{voisin}}, y_{\text{courant}} - y_{\text{voisin}})$$

3. Le produit scalaire dot entre le vecteur de déplacement normalisé V_{norm} et le vecteur du vent normalisé W_{norm} est calculé :

$$\text{dot} = v_{\text{norm}} \cdot w_{\text{norm}}$$

4. Le poids final attribué au voisin est déterminé par la formule :

$$\text{Weight} = \max(0.0, 1.0 + s \times \text{dot})$$

Un vent aligné dans la direction Voisin -> Cellule courante génère un poids > 1.0 , maximisant le transfert de pollution, tandis qu'un vent opposé freine cette propagation.

C. Comportement et lois de transition spécifiques des cellules

Le système met en œuvre quatre types de cellules spécialisées, possédant chacune des règles de calcul propres :

- **Cellule d'Air (AIR)** : Agent central de la diffusion atmosphérique. Les obstacles physiques de type Bâtiment y sont totalement ignorés. La cellule calcule la moyenne pondérée de la pollution de ses voisins éligibles (avg) via l'impact du vent :

$$avg = \frac{\sum (Pollution_{voisin} \times Weight_{voisin})}{\sum Weight_{voisin}}$$

Elle applique ensuite le taux de diffusion global D (diffusionRate) :

$$P_{diffusé} = P_t + D \times (avg - P_t)$$

Enfin, elle subit l'absorption cumulée de toutes les cellules de végétation environnantes (V) (selon le taux d'absorption global A) avant d'établir sa pollution finale :

$$P = \max \left(0.0, P_{diffusé} - \sum_v (A \times 0.5 \times customRate_{voisin}) \right)$$

(customRatevoisin étant la "puissance d'absorption" de la case végétation i)

- **Cellule de Végétation (VEGETATION)** : Agit comme un filtre vert absorbant. Elle réduit localement son propre taux de pollution à chaque pas de temps en fonction de sa capacité individuelle (customRate):

$$P = \max (0.0, P_t - (A \times customRate))$$

- **Cellule d'Usine (FACTORY)** : Source industrielle et continue d'émissions de polluants (ex. dioxyde de soufre). Son activité varie de façon cyclique selon l'heure de la journée, déterminée par le temps écoulé (1 tick de simulation = 1 minutes) donc

$$Heure = \left\lfloor \frac{tick}{60} \right\rfloor \pmod{24}$$

custom Rate représente la puissance de l'usine

- En journée (de 07:00 à 21:59) : La pollution émise est égale à son customRate.
- De nuit (de 22:00 à 06:59) : La production est réduite en raison d'une baisse d'activité :

$$P = 0.8 \times customRate$$

- **Cellule de Bâtiment (BUILDING)** : Obstacle urbain imperméable représentant les structures en béton ou les habitations. Son niveau de pollution reste bloqué de manière absolue à 0.0. Sur le plan physique, il bloque net la propagation des flux d'air pollué, forçant les cellules d'air adjacentes à l'exclure de leurs calculs de moyenne.

2. Mécanisme de mise à jour

Dans un automate cellulaire, l'état futur d'une cellule dépend de son état actuel et de celui de ses proches voisines. il est donc important de prévoir cette logique dans le système de mise à jour de notre automates cellulaires afin d'éviter les cas où si l'application mettrait à jour les valeurs de pollution au fur et à mesure du parcours de la grille (de gauche à droite et de haut en bas), les premières cellules modifiées fausserait les calculs des cellules suivantes au cours du même tour. La pollution se propageait alors artificiellement plus vite dans le sens de lecture de la boucle.

Pour éliminer ce genre de bug et garantir que toutes les cellules évoluent de manière parfaitement synchrone, chaque case de la grille gère deux variables d'état distinctes :

- L'état courant (pollutionLevel) : C'est la valeur physique validée pour le tick actuel. Elle est fixe, sert de base de calcul pour les cellules voisines et est utilisée par la vue pour l'affichage graphique.
- L'état futur (nextPollutionLevel) : C'est une variable tampon temporaire. Elle stocke le résultat des calculs en cours pour le pas de temps suivant, restant totalement invisible pour le reste de la grille tant que le tour n'est pas achevé.

Le passage du temps (méthode tick() de l'objet Simulation) est ainsi orchestré par le Modèle en deux étapes strictes :

1. La phase de calcul global : Le moteur parcourt l'intégralité de la grille. Chaque cellule évalue son comportement (diffusion, émission, absorption) en lisant l'état courant de son voisinage, puis enregistre son résultat dans son état futur.
2. La phase de validation globale (Commit) : Une fois tous les calculs de la grille finalisés, une seconde boucle rapide copie la valeur de l'état futur dans l'état courant de chaque cellule. La transition est ainsi simultanée pour toute l'application.

Cette phase de calcul est faite de manière synchrone : le calcul de l'état future se fait sur chaque cellule l'une après l'autre de même pour la mise à jours des etat (la phase de "commit") mais on pourrait imaginer plus tard une mise à jours en parallèle des cellule (et attendre la mise à jours de toute les cellule avant de passé a la phase de commit) ce qui nous ferait gagner en temps d'exécutions sur le calcule des mise à jours

3. Ajout simplifié de type de cellule

Notre moteur de simulation prend plein avantage du langage java avec la POO, en centralisant les caractéristiques communes de toutes les cellules de la grille au sein d'une unique Abstraite : AbstractCell.

Celle ci possède les attribue / méthode suivante:

- La position géométrique fixe : Les coordonnées immuables (x, y) de la cellule sur la grille bidimensionnelle.
- Les variables de double-buffering : Le niveau de pollution courant (pollutionLevel) et le niveau de pollution futur (nextPollutionLevel).
- Les attributs physiques internes : Un facteur d'activité individuel (customRate) permettant d'introduire des variations de comportement d'une cellule à l'autre (par exemple, une efficacité d'absorption variable pour la végétation, ou la puissance d'une usine).
- La logique de validation : La méthode commitState() qui effectue la copie de l'état futur vers l'état courant à la fin de chaque tick.

Ce qu'elle permet dans l'application :

- La mise en œuvre du polymorphisme : L'objet grille manipule un tableau bidimensionnel AbstractCell[[]]. Cela permet au moteur de simulation d'ordonner l'évolution de la grille et le calcul des états en appelant la méthode computeNextState() de manière uniforme, sans devoir tester ou connaître le type exact (Air, Usine, Bâtiment, Végétation) de la case traitée.
- L'ajout direct de nouveaux types de cellules : La classe définit les méthodes abstraites getName() et computeNextState(). Pour intégrer un nouvel élément dans la simulation (par exemple, une route ou un plan d'eau), il suffit de créer une nouvelle classe fille qui hérite d'AbstractCell et d'y coder ses règles de calcul spécifiques. Cette intégration se réalise de manière isolée, sans impacter les classes du moteur de calcul ou des contrôleurs.

V. Conclusion

Le projet CyBreathe a permis de concevoir une application fonctionnelle de simulation de pollution atmosphérique en 2D, répondant avec succès aux attentes du cahier des charges. Grâce à une répartition claire des rôles et à l'utilisation d'outils collaboratifs (GitHub, Gradle), l'équipe a pu développer un automate cellulaire robuste et modulaire.

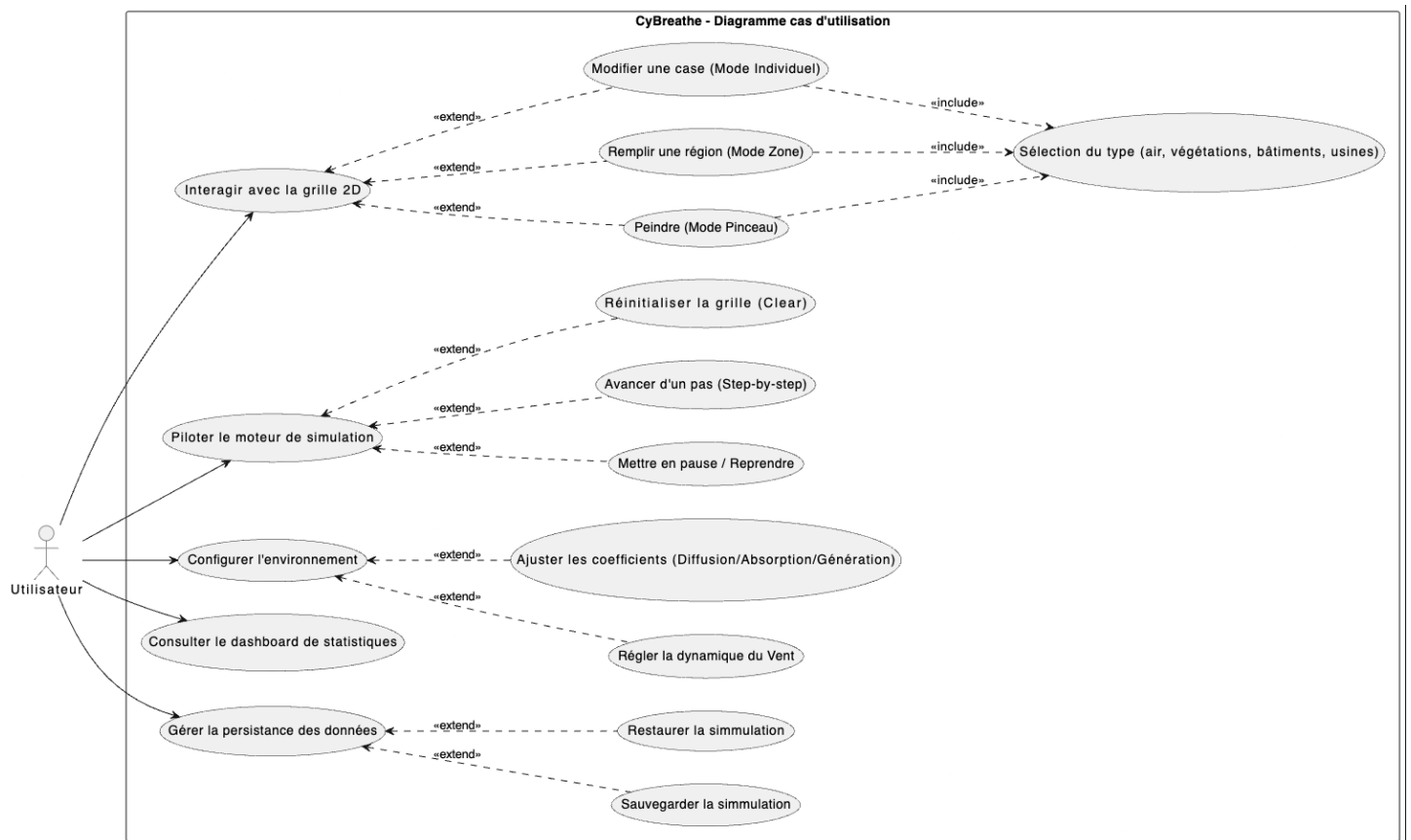
Sur le plan technique, le choix de l'architecture MVC s'est révélé être un atout majeur. Ce découpage a permis d'isoler complètement la logique métier, facilitant la mise en place de la double interface graphique (JavaFX) et textuelle (CLI), tout en garantissant une haute testabilité via les tests unitaires. De plus, l'implémentation du double tampon (`computeNextState` / `commitState`) a résolu efficacement le problème des biais de propagation liés au sens de lecture de la grille.

Bien que la simulation présente des limites physiques évidentes comme le comportement parfait des bâtiments, l'absorption infinie de la végétation, l'absence d'échelle métrique réelle ou le vent unidirectionnel —, le code reste hautement évolutif. L'utilisation du polymorphisme via la classe `AbstractCell` permet d'ajouter de nouveaux types de cellules très simplement, sans impacter le moteur de calcul.

En guise d'ouverture, plusieurs perspectives d'amélioration sont envisageables. À court terme, la phase de calcul des états futurs pourrait être parallélisée pour optimiser le temps d'exécution sur des grilles plus grandes. À long terme, l'introduction d'une échelle physique concrète et de turbulences aérodynamiques permettrait de rapprocher CyBreathe d'un véritable outil d'aide à la décision pour l'urbanisme éco-responsable.

VI. Annexes

1. Diagramme de cas d'utilisation



2. Diagramme de classe

